DO NOW — RECALL

Programming paradigms

Name: _____ Class: _____ Date: _____

1. What is an object?

2. What does a constructor do?

3. What does a return statement do?

4. What property distinguishes a pure function from a method that changes a shared counter?

DO NOW — CHECK / OBJECTIVES

Correct and improve

Check page 1 in GREEN PEN. Write corrections in full.

1. Model answer

A particular instance of a class.

2. Model answer

Initialises a new object.

3. Model answer

Ends the function and passes a result to its caller.

4. Model answer

A pure function has no externally observable side effects and returns the same result for the same inputs.

Vocabulary — know these terms

paradigm — Overall style for organising programs.

procedural — Organises a program around sequences of operations and subroutines.

object-oriented — Encapsulates state and behaviour into objects.

functional — Emphasises functions applied to values and reduced side effects.

declarative — Expresses desired results rather than imperative step sequences.

WE DO — SOURCE CHECK / IMPROVE

Only use this section AFTER attempting the We Do, You Do and Exit Ticket. Compare with the original classroom lesson's CHECK explanation. Correct errors in GREEN PEN and explain why.

Guided 1 — original CHECK

Model response

This is object-oriented design; after Add(2), its Points becomes 7.

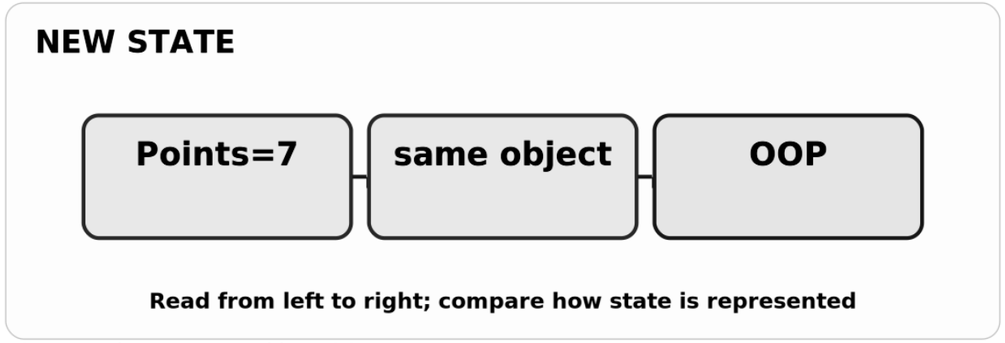
IMPROVE: Explain one correction to your own reasoning.

We Do Check · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

Classify a Score object storing Points and updating it using Add(2).

The Score object holds Points alongside a method that changes its state.



NEW STATE; Points=7; same object; OOP

NEW LEARNING

Read, annotate and explain

Procedural

Instructions and procedures operate on arguments or shared data. A function can be reused independently.

Object-oriented

The Account object combines state (Balance) and behaviour (Deposit). With inheritance, a subclass can reuse and specialise a class. A virtual method overridden in a derived class supports polymorphism: the actual object decides which implementation runs.

Functional style

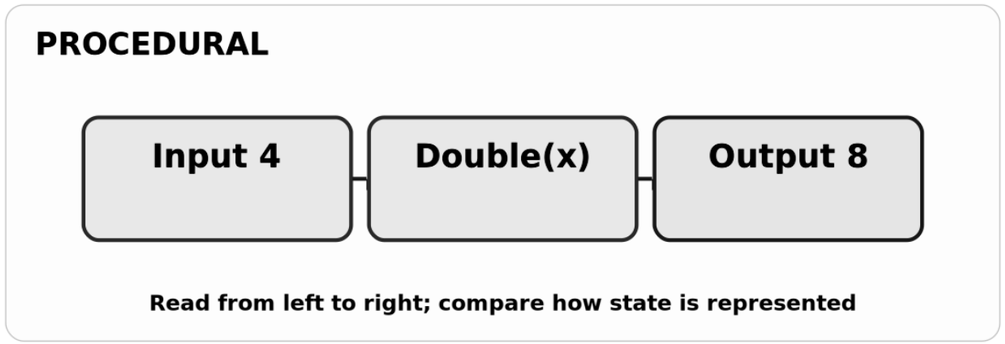
Values are transformed using expressions/functions, often emphasising avoiding mutation of existing input data.

Explain one step in your own words, using the correct technical terms.

New Learning · original lesson diagrams

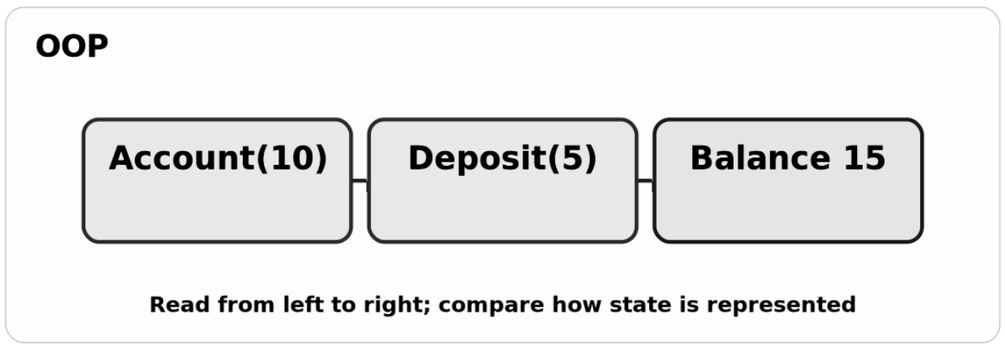
Diagrams appear after the related explanation. Annotate the original source visual.

New Learning



PROCEDURAL; Input 4; Double(x); Output 8

New Learning



OOP; Account(10); Deposit(5); Balance 15

EXIT TICKET / REFLECTION

Check your understanding

Complete the actual lesson Exit Ticket before checking the answer.

A language such as C# can support both procedural and object-oriented programming.

How confident are you? Circle one: Confident / Getting there / Need help

Explain your choice, identifying one concept you can now use independently.

GREEN PEN — Correct and explain

Choose one mistake from your independent work. Write the correct method and explain precisely why your original answer was wrong.

Next step: revisit the model or a relevant revision-guide section if needed.

YOU DO — INDEPENDENT PRACTICE

Questions · Part 2

Answer independently. Show code, calculations, a trace or justified reasoning as appropriate.

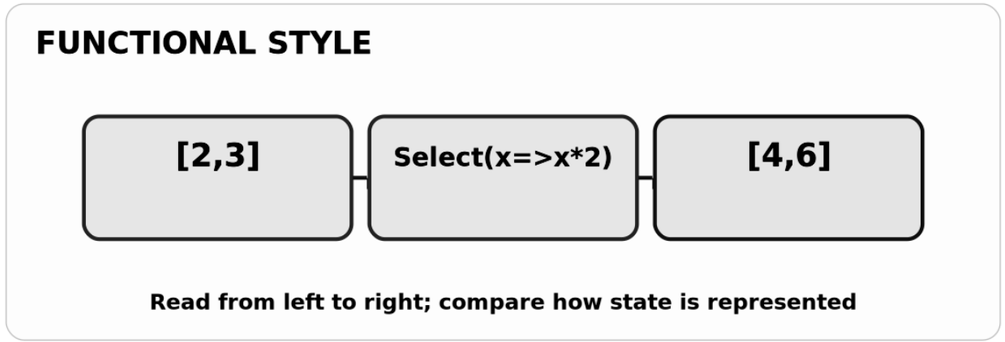
Question 3

Explain why `values.Select(x=>x*2)` can be considered functional-style programming, and one limitation of labelling C# purely functional.

New Learning · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

New Learning



FUNCTIONAL STYLE; [2,3]; Select(x=>x*2); [4,6]

I DO — WORKED EXAMPLE

Follow the worked method

Classify a function Square(x) that returns x*x without storing object state.

Predict

It is procedural when used as a standalone function in a procedural programme.

Model response

Square(3) returns 9; it does not change object state. C# can use more than one paradigm.

Follow the declared and actual types

Both variables are declared as BaseMessage . Their actual objects are EmailMessage and TextMessage , each inheriting from the base class.

Why is virtual important?

The base method is virtual and both derived classes override it. The language dispatches to the actual object's implementation.

Output and design consequence

Outputs Email then Text . The caller can add another derived message class without writing separate type-checking branches.

YOU DO — INDEPENDENT PRACTICE

Questions · Part 1

Answer independently. Show code, calculations, a trace or justified reasoning as appropriate.

Question 1

Classify a program using standalone functions with arguments and returns. Give one advantage.

Question 2

Compare procedural and object-oriented account implementations, then explain how an overridden method on a derived SavingsAccount can be called through an Account reference.

WE DO — TRY · DISCUSS · IMPROVE

TRY: Use the original guided questions and diagrams in the preceding We Do pages. DISCUSS: Explain and challenge methods before independent practice. IMPROVE: Note one change now; compare with the source model only AFTER the Exit Ticket.

Guided 1: Predict The Score object holds Points alongside a method that changes its state.

DISCUSS — Compare your methods. Justify one important decision, identify one difference, and agree a specific improvement.

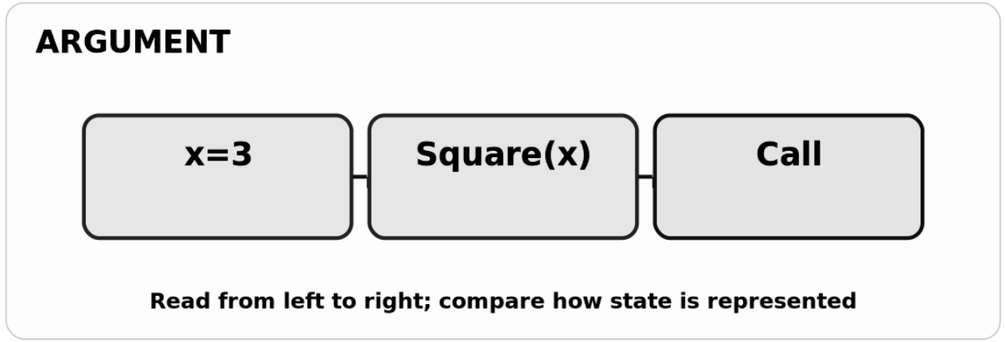
IMPROVE — Write one precise correction or stronger explanation in your own answer.

I Do · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

Classify a function Square(x) that returns x*x without storing object state.

It is procedural when used as a standalone function in a procedural programme.



ARGUMENT; x=3; Square(x); Call

Classify a function Square(x) that returns x*x without storing object state.

It is procedural when used as a standalone function in a procedural programme.

CALCULATE; 3 × 3; return 9; No object state

WE DO — GUIDED PRACTICE

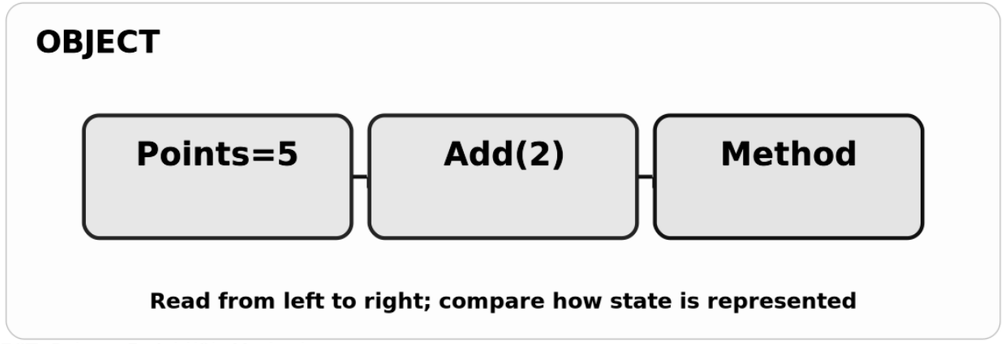
Try → Discuss → Improve

We Do · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

Classify a Score object storing Points and updating it using Add(2).

The Score object holds Points alongside a method that changes its state.



OBJECT; Points=5; Add(2); Method